

Three SAT Problems

NANDy's Logic Lab · The Great Cube · The Treaty

Question 1: NANDy's Logic Lab

NANDy is an inventor with one conviction: any circuit worth building can be built out of NAND gates alone. NANDy also has no patience for waste — every gate costs money.

You are given a Boolean formula in **CNF**. Build a combinational circuit that outputs **True** exactly when the formula is satisfied, using **only NAND gates**, and using **as few of them as mathematically possible**.

The rules

- Every gate is a NAND gate with **arbitrary fan-in** — any number of inputs, at least one. A NAND with a single input is a NOT gate.
- Every gate reads only primary inputs and **strictly earlier** gates, so the circuit is acyclic.
- The last gate is the output, and on **every one of the 2^n assignments** to $x_1 \dots x_n$ its value equals the value of the CNF.
- The gate count is the **minimum over all such circuits**.

What you write

One function, `synthesize(cnf)`. Its input is the CNF as a list of lists of integers ($+i$ for x_i , $-i$ for $\neg x_i$); its output is the circuit as a nested list, where element k lists the node ids feeding gate k .

For `cnf = [[-1, 2]]`, the formula $(\neg x_1 \vee x_2)$, a correct minimal answer is `[[1], [0, 2]]`:

```
G1 = NAND(x2, x2)  fan-in 1, so this is  $\neg x_2$ 
G2 = NAND(x1, G1)   $\neg(x_1 \wedge \neg x_2) = \neg x_1 \vee x_2$ 
```

One gate is provably not enough, so 2 is the minimum.

Two things to be careful about

Minimality is the whole question. A correct circuit with one gate too many scores the same as a wrong one. You need a search that *proves* no smaller circuit exists, not a construction that happens to be small.

You must model this in Z3. Enumerating circuits in Python, hand-crafting them from the clause structure, or hard-coding answers scores zero — even when the returned circuit is correct and minimal.

Full specification: everything you actually need to write the code — exact input and output formats, encodings, worked examples, checker and test instructions — is in `NANDy/README.md`. Read it before you start. This document is only a summary.

Question 2: The Great Cube

Tamanna is being mind-controlled by Jean Grey, and the only way out is to solve the cube on the table in front of her. She cannot: solving a cube is a *loop* — turn it, look at it, decide, turn again — and Jean lives in the “decide”.

So Tamanna decides nothing. Her friend Peter works out the **entire sequence of turns in advance**, and she executes it blind, with no feedback and nothing for Jean to read. There is a deadline of T minutes, one turn per minute.

Peter cannot solve a Rubik’s cube either. Peter can write down a formula that is satisfiable exactly when a solution exists, and hand it to a CDCL solver.

The problem

Given a scrambled $3 \times 3 \times 3$ cube and a deadline T , produce a sequence of at most T face turns that leaves every face a single colour — or prove that no such sequence exists.

The whole reason this is a SAT problem is that **the sequence is fixed in advance**. You cannot look at the cube after move 3 and decide move 4. So you cannot search the way a human does: you must write a formula whose variables *are* the entire plan — the state of all 54 stickers at all $T + 1$ moments, and which turn happens at each of the T steps — such that the formula is satisfiable if and only if a T -move solution exists.

This shape of problem is called **bounded planning**. A satisfying assignment *is* Tamanna’s instruction list, and if the solver reports UNSAT that is not a failure: it is a *proof* that the cube cannot be solved in T minutes.

The moves

Eighteen in total: each of the six faces U D F B L R turned 90° clockwise (a bare letter), 90° counter-clockwise (') or 180° (2), where clockwise means clockwise as seen looking at that face from outside. Every move costs one minute, half turns included. Nothing else is legal.

What you write

One function, `solve(cube, T)`. The cube arrives as a list of six single-key dictionaries, one per face: the key is the colour of that face’s *centre* sticker — which is what names the face, since centres never move — and the value is the list of the eight stickers around it. You return the list of move names, at most T of them, or -1 if the cube cannot be solved in time.

The colour-to-face mapping, the orientation each face is read in, and the exact sticker ordering are all specified in the README, together with a fully worked example you can check your parsing against.

Two things to be careful about

Everything must be propositional. Only Boolean variables and Boolean connectives — no integers, no arithmetic, no theory reasoning. The solver underneath is CDCL, and keeping the encoding purely propositional is what lets it do its job.

A turn moves 20 stickers and leaves 34 alone. Saying what moves is only half the job. If you never say the other 34 stay put, the solver is free to rearrange them, and it will.

Full specification: everything you actually need to write the code — exact input and output formats, encodings, worked examples, checker and test instructions — is in `rubik/README.md`. Read it before you start. This document is only a summary.

Tools provided

- `check.py` — an independent checker. It does not use Z3: it replays your move sequence one turn at a time and reports whether the cube ends up solved.
- `visualizer.py` — watch the encoding work. It shows the solver ruling out each depth in turn, then redraws the cube after every move of the plan it finds.
- `tests/` — sample cubes, each with its own deadline T , plus `make_test.py` for building your own.

Question 3: The Treaty

There are N queens and one hall, and the hall is an $N \times N$ grid of tiles. The queens are not on speaking terms; nobody remembers exactly why. The treaty that ended it is one sentence long: **no queen may be able to see another one**. A queen sees along her row, along her column, and along both diagonals, and she sees all the way to the wall.

All N of them have to stand somewhere. A few have already picked their tile and are not moving. A few tiles are off limits — that is where the last argument happened.

The problem

Place exactly N queens on an $N \times N$ board so that

- no two share a **row**,
- no two share a **column**,
- no two share a **diagonal** — either direction, any length, i.e. no two cells with $|r_1 - r_2| = |c_1 - c_2|$,
- every cell in `placed` holds a queen,
- no cell in `blocked` holds a queen.

Report a placement, or report that none exists.

What you write

One function, `solve(n, placed, blocked)`. The board is $n \times n$ with rows and columns numbered $0 \dots n - 1$ and $(0, 0)$ the top-left corner; `placed` and `blocked` are lists of cells that must, and must not, hold a queen.

You return a list of n column indices — element r is the column of the queen in row r — or -1 if no placement exists. Since no two queens share a row and there are n of them on n rows, every row holds exactly one, which is why one column index per row describes the whole board.

`solve(4, [], [])` may return `[1, 3, 0, 2]`:

```
row 0: . Q . .  
row 1: . . . Q  
row 2: Q . . .  
row 3: . . Q .
```

The 4×4 board has exactly two solutions, $[1, 3, 0, 2]$ and its mirror $[2, 0, 3, 1]$; either is accepted. $N = 2$ and $N = 3$ have none, so `solve` returns -1 for them.

Two things to be careful about

Keep it propositional. Only Boolean variables and Boolean connectives. In particular, do not reach for `Sum(If(cell, 1, 0)) == n` to count the queens — the count follows from the row and column rules on its own, and working out why is part of the exercise.

Both diagonals, every length. It is easy to encode the two main diagonals and forget the short ones near the corners. Every pair of cells with $|r_1 - r_2| = |c_1 - c_2|$ is a conflict, however close to the edge.

Full specification: everything you actually need to write the code — exact input and output formats, encodings, worked examples, checker and test instructions — is in `N_queens/README.md`. Read it before you start. This document is only a summary.